1. Introdução

O GeekVerse é um sistema desenvolvido em Java para auxiliar no gerenciamento de participantes de um evento com temática geek. A aplicação funciona por meio de um menu no console e permite cadastrar participantes de diferentes categorias, consultar os inscritos, visualizar a quantidade total de participantes e exportar os dados para um arquivo de texto.

O projeto foi estruturado com base nos princípios da Programação Orientada a Objetos (POO), utilizando classes, objetos, encapsulamento, herança, abstração, polimorfismo, interface e tratamento de exceções.

2. Objetivo do sistema

O objetivo do GeekVerse é organizar informações básicas dos participantes de um evento, mantendo os dados em uma coleção durante a execução do programa e oferecendo operações simples de cadastro, consulta, listagem e exportação.

3. Funcionalidades

- **Cadastro de participantes:** Permite inserir nome, e-mail, categoria de participação e atividade.
- **Validação de dados:** Verifica campos obrigatórios e formato básico do e-mail. Também impede e-mails duplicados no evento.
- **Listagem:** Exibe os dados dos participantes cadastrados, incluindo observações específicas de cada categoria.
- **Busca por tipo:** Permite localizar participantes conforme a categoria informada.
- **Contagem:** Apresenta a quantidade total de participantes cadastrados.
- **Exportação:** Grava os dados dos participantes em um arquivo de texto no formato UTF-8.

4. Organização dos arquivos

O código-fonte está organizado dentro da pasta src, separado em pacotes:

Pacote	Arquivos e responsabilidades
main	Main.java — ponto de entrada do programa.
model	Pessoa, Usuario, Participante, Gamer, Cosplayer, Palestrante, Visitante, Evento e Exibivel.
service	MenuConsole.java — interação com o usuário por meio do menu no console.
exception	DadosInvalidosException.java — exceção personalizada para dados inválidos.

5. Descrição das principais classes

- **Usuario —** Classe abstrata que representa um usuário genérico. Possui um identificador gerado automaticamente e o método abstrato getPerfil().
- **Pessoa —** Herda de Usuario e armazena nome e e-mail. Valida os dados por meio dos métodos setters e implementa Exibivel.
- **Participante —** Herda de Pessoa e acrescenta o tipo de participação e a atividade. Define constantes para as categorias.
- **Gamer —** Especialização de Participante para pessoas inscritas em atividades de jogos. Sobrescreve exibirDados() para apresentar uma observação própria.
- **Cosplayer —** Especialização de Participante para cosplayers. Sobrescreve exibirDados() com uma orientação específica.
- **Palestrante —** Especialização de Participante para quem ministra palestras ou oficinas. Sobrescreve exibirDados().
- **Visitante —** Especialização de Participante para visitantes do evento. Sobrescreve exibirDados().
- **Evento —** Armazena os dados do evento e a lista de participantes. Realiza cadastro, busca, contagem, listagem e exportação.
- **Exibivel —** Interface que declara o método exibirDados(), utilizado para padronizar a apresentação de informações.
- **MenuConsole —** Apresenta as opções do sistema e encaminha as ações escolhidas pelo usuário.
- **DadosInvalidosException —** Exceção personalizada utilizada para sinalizar dados inválidos durante operações do sistema.

6. Conceitos de Programação Orientada a Objetos

- **Encapsulamento:** Os atributos das classes são privados e o acesso aos dados ocorre por métodos públicos, como getters e setters. Os setters também realizam validações.
- **Herança:** Pessoa herda de Usuario; Participante herda de Pessoa; e Gamer, Cosplayer, Palestrante e Visitante herdam de Participante.
- **Abstração:** Usuario é uma classe abstrata e define uma estrutura comum para seus tipos derivados.
- **Polimorfismo:** As subclasses de Participante sobrescrevem exibirDados(). Assim, uma referência do tipo Participante pode executar a versão correspondente ao objeto real.
- **Interface:** Exibivel define um contrato para classes que precisam apresentar seus dados por meio do método exibirDados().
- **Sobrescrita:** O uso de @Override indica a redefinição de métodos herdados, como exibirDados() e getPerfil().
- **Coleções:** Evento utiliza uma lista baseada em ArrayList para armazenar os participantes durante a execução.
- **Tratamento de exceções:** DadosInvalidosException permite comunicar situações inválidas, como nome vazio, e-mail inválido ou duplicado.

7. Relação entre as classes

A estrutura principal de herança pode ser representada da seguinte forma:

```
Usuario (classe abstrata)
└── Pessoa (implementa Exibivel)
    └── Participante
        ├── Gamer
        ├── Cosplayer
        ├── Palestrante
        └── Visitante

Evento 1 ─── 0..* Participante
Evento mantém uma coleção de participantes.
```

8. Como executar o programa

O projeto pode ser compilado e executado pelo terminal do PowerShell, estando na pasta raiz geekverse. Os comandos abaixo consideram a estrutura de pastas utilizada no projeto.

```
New-Item -ItemType Directory -Force bin
javac -encoding UTF-8 -d bin (Get-ChildItem -Recurse -Path src -Filter *.java | ForEach-Object { $_.FullName })
java -cp bin main.Main --menu
```

Após iniciar, o usuário pode escolher as opções exibidas no menu. A opção de encerramento finaliza a aplicação.

9. Arquivo de exportação

A funcionalidade de exportação grava os participantes em um arquivo de texto chamado participantes.txt. O conteúdo é escrito com codificação UTF-8 para preservar caracteres acentuados. O arquivo é gerado na pasta de execução do programa.

10. Verificações realizadas

Durante a verificação do projeto, a compilação dos arquivos Java foi executada sem apresentar erros. Também foi executada a demonstração do sistema, que cadastrou participantes de diferentes categorias, verificou situações de dados inválidos, realizou uma busca e gerou o arquivo participantes.txt. O menu interativo também foi iniciado e utilizado para cadastrar um participante e consultar a quantidade total cadastrada.

Essas verificações registram os testes efetivamente realizados durante o desenvolvimento. Outras combinações de entrada podem ser testadas conforme a necessidade.

11. Conclusão

O desenvolvimento do GeekVerse permitiu aplicar conceitos fundamentais de Programação Orientada a Objetos em uma aplicação Java. A separação das responsabilidades em classes e pacotes facilita a organização do código, enquanto a herança, o polimorfismo, a abstração e o

encapsulamento contribuem para uma estrutura reutilizável e de fácil manutenção. O sistema reúne funcionalidades básicas de gerenciamento de participantes e exportação de informações.